

classi e oggetti

flavio

May 11, 2026

Contents

1	Classi e oggetti	1
1.1	classe e oggetti a confronto	1
1.2	Classe e file sorgenti	1
1.2.1	file Automobile.java	2
1.3	Librerie di Classi e package	2
1.4	Struttura di una classe	2
1.5	Campi	3
1.5.1	Dichiarazione di un campo	3
1.5.2	variabili locali vs campi	3
1.6	Metodi	3
1.6.1	definizione metodo	3
1.7	Costruttori	3
1.7.1	Costruttore per la classe Counter	4
1.7.2	Creazioni oggetti	4
2	Incapsulamento	4
2.1	Definizione	4
2.2	Interazione tra classi	4
2.3	Accesso a campi e metodi	4
2.3.1	privati	5
2.3.2	pubblici	5
3	Approfondimento sui metodi	5
3.1	metodi statici	5
3.2	Fun fact su main()	5
3.3	Setter e Getter	5

4	Campi statici	6
4.1	campi statici noti	6
4.2	importazione statica dei campi	6
5	Enumerazioni	6
5.1	Dichiarazione Enumerazione	6
5.2	Con Costruttori, Campi, Metodi (acchittata)	7
5.3	values e valueOf	7
5.4	Enumerazioni con switch	8
6	Classi Wrapper	8
6.1	Confrontare oggetti	8
6.2	Autoboxing e auto-unboxing	9
6.2.1	Nelle stringhe	9
6.3	Costo oggetto memoria	9

1 Classi e oggetti

Una classe è una parte del codice sorgente che descrive un particolare tipo di oggetto. Sono definite dal programmatore. La classe definisce la struttura in termini di: **-campi** (stato) degli oggetti **-metodi** degli oggetti -Un oggetto è un'istanza (un esemplare) di una classe

1.1 classe e oggetti a confronto

Classe

Definita all'interno del codice sorgente del programma

scritta dal programmatore

Specifica il comportamento dei suoi oggetti mediante il codice dei metodi

Oggetto

un'entità all'interno di un programma

creato quando un programma viene eseguito

Si comporta nel modo predefinito

1.2 Classe e file sorgenti

- Ogni classe è memorizzata in un file separato il cui nome è quello della classe
- Il nome del file **Deve essere uguale** a quello della classe, con estensione .java
- I nomi iniziano sempre con una maiuscola
- I nomi in Java sono case-sensitive

1.2.1 file Automobile.java

```
public class Automobile
{
    ...
}
```

1.3 Librerie di Classi e package

I programmi Java non sono scritti da zero. Esistono migliaia di classi di libreria per ogni esigenza. Un package “speciale” è `java.lang`: contiene le classi fondamentali per la programmazione in Java.

1.4 Struttura di una classe

```
public class Counter
{
    // campi
    private int value;

    //costruttore
    public Counter()
    {
        value = 0;
    }
    public Counter(int value){
        this.value = value;
    }
    //metodi

    //incrementa il contatore
    public void count() {
        value++;
    }
    // ottiene il valore corrente del contatore
    public int getValue() {return value;}
    public static void main(String[] args)
    {
        Counter a = Counter();
    }
}
```

}

1.5 Campi

Un campo (detto anche **variabile di istanza**) costituisce la memoria privata di un oggetto; ha un tipo di dato o un nome/identificativo.

1.5.1 Dichiarazione di un campo

se sono tra parentesi quadre sono opzionali

```
private [static] [final] tipo_di_dati nome;
```

1.5.2 variabili locali vs campi

- Sono visibili almeno all'interno di tutti gli oggetti della stessa classe. Esistono per tutta la vita di un oggetto
- Le variabili locali sono variabili definite all'interno di un metodo e sono visibili all'interno del metodo e la loro durata è limitata alla durata dell'esecuzione del metodo.

1.6 Metodi

Il nome di un metodo inizia con una lettera minuscola, mentre le parole seguenti iniziano con una lettera maiuscola: convenzione detta CamelCase.

1.6.1 definizione metodo

Un metodo è diviso in 2 parti: Intestazione e corpo. **L'intestazione** comprende la visibilità del metodo, il tipo di ritorno e il nome del metodo. Il corpo comprende le istruzioni.

```
#+begin_src java public tipo_di_dati nomeDelMetodo(tipo_di_dati nomeParam1, ..., tipo_di_dati nomeParam) { istruzione 1; istruzione m; } #+end_src java
```

1.7 Costruttori

I costruttori sono metodi per la creazione di oggetti. Hanno lo stesso nome della classe; il loro scopo è inizializzare i campi dell'oggetto. Possono prender zero, uno o più parametri. NON hanno valori di uscita **ma non viene specificato void**. Una Classe può avere più costruttori o non averne nessuno che differiscono nel numero e nei tipi dei parametri. Se non viene specificato

alcun costruttore allora java crea una classe con costruttore vuoto senza parametri. Inizializza le variabili d'istanza ai valori di default

1.7.1 Costruttore per la classe Counter

```
public Counter()
{
    value = 0;
}
public Counter(int initialValue)
{
    value = initialValue;
}
```

1.7.2 Creazioni oggetti

2 Incapsulamento

L'incapsulamento è una delle idee chiave della programmazione ad oggetti. Permette di nascondere le informazioni all'utente (**information hiding**).

2.1 Definizione

Il processo che nasconde i dettagli realizzativi rendendo pubblica un'interfaccia.

- dettagli realizzativi: campi e implementazione
- interfaccia pubblica: metodi pubblici

L'obiettivo è creare una scatola nera così che l'utente non sappia tutto, ma solo lo stretto necessaria per usare il programma. **Facilità il lavoro di gruppo** e l'aggiornamento del codice. Aiuta a rilevare errori: in presenza di moltissime classi (perché?)

2.2 Interazione tra classi

Le classi interagiscono con le altre principalmente attraverso **costruttori e metodi pubblici**. Le altre classi **non DEVONO** conoscere i dettagli implementativi di una classe per usarla in modo efficace.

2.3 Accesso a campi e metodi

I campi e metodi possono essere pubblici e privati.

2.3.1 privati

Solo la classe che li ha creati può accedere ai campi e chiamare i metodi

2.3.2 pubblici

I metodi possono essere chiamati da tutti e i campi possono essere visti da tutti.

3 Approfondimento sui metodi

I metodi permettono di modularizzare un programma separandone i compiti in unità autocontenute. Le istruzioni di un metodo non sono visibili da un altro metodo. Certi metodi non utilizzano lo stato dell'oggetto.

3.1 metodi statici

Il modificatore statico specificato nell'intestazione del metodo. Due modi per chiamare:

- Dall'interno della classe, semplicemente chiamando il metodo.
 - Dall'esterno, NomeClasse.nomeMetodo()

```
public static String getLinea(int k)
{
    String s = "";
    while(k-- > 0) s+="*";
    return s;
}
```

3.2 Fun fact su main()

Perchè è statico? la *J*ava *V*irtual *M*achine (*JVM)invoca il metodo main della classe specificata prima. La classe potrebbe non avere un costruttore senza parametri con cui creare l'oggetto e senza static verrà creata una istanza.

3.3 Setter e Getter

L'accesso ad alcuni campi è garantito dai metodi getX() setX() dove X è il campo. Garantiscono la consistenza dei dati. Fanno da filtro tra i dettagli implementativi e ciò che vede l'utente esterno.

4 Campi statici

Specificando `static` nella dichiarazione del campo. Si può accedere dall'interno della classe semplicemente con il `nomeCampo`. `NomeClass.nomeCampo`.

4.1 campi statici noti

Sono tutti `public final static` perchè accessibili a tutti; `final` perchè costanti; `static` perchè non variano secondo lo stato dell'oggetto.

4.2 importazione statica dei campi

`Import static` permette di importare campi `static` come se fossero definiti nella classe in cui si importano.

```
import static java.lang.Math.E;
public class StaticImport
{
    public static void main(String[] args)
    {
        System.out.println(E);
    }
}
```

5 Enumerazioni

Le enumerazioni sono dei tipi i cui valori possono essere scelti tra un insieme predefinito di identificatori univoci. Ogni identificatore corrisponde a una costante. Tutte le costanti enumerative sono implicitamente `static`. Non si possono creare oggetti di tipo enumerato.

5.1 Dichiarazione Enumerazione

```
public enum SemeCarta
{
    CUORI,
    QUADRI,
    FIORI,
    PICCHE
}
```

5.2 Con Costruttori, Campi, Metodi (acchittata)

Senza enumerazioni

```
public class Mese
{
    private int mese;
    public Mese(int mese) {this.mese = mese;}
    public int toInt() {return mese;}
    public String toString()
    {
        switch(mese)
        {
            case 1: return "GEN";
            case 2: return "FEB";
            case 12 return "DIC";
            default: return null;
        }
    }
}
```

Con enumerazioni

```
public enum Mese
{
    GEN(1),FEB(2),MAR(3),APR(4),MAG(5),GIU(6),LUG(7),AGO(8),SET(9),OTT(10),NOV(11)
    private int mese;
    Mese(int mese) {this.mese = mese;}
    public int toInt() {return mese;}

    public static void main(String[] args)
    {
        Mese[] mesi = SemeCarta.valueOf
        for(int i = 0; )
    }
}
```

5.3 values e valueOf

Per ogni enumerazione, il compilatore crea due metodi statici `values()` che restituisce un array delle costanti enumerative. Viene generato anche un

metodo `ValueOf(String NomeCostante)` che restituisce la costante enumerativa associata alla stringa fornita in input. Se il valore non esiste, viene generato un errore(eccezione)

5.4 Enumerazioni con switch

```
Semecarta seme null
switch(seme)
{
    case GEN : System.out.println("NUOVO ANNO");break;
    default: break;
}
```

6 Classi Wrapper

Permettono di gestire un valore primitivo come un oggetto. Forniscono metodi di accesso e visualizzazione dei valori. Essi sono

Tipo primitivo	Classe	Argomenti del costruttore
byte	Byte	byte o String
short	Short	short o String
int	Integer	int o String
long	Long	long o String
float	Float	float, double o String
double	Double	double o String
char	Character	char
boolean	Boolean	boolean o String

6.1 Confrontare oggetti

Dato che adesso abbiamo oggetti ci troviamo in una situazione simile alle stringhe Per il confronto usiamo:

- `equals()`: restituisce true se e solo se l'oggetto in input è

un intero di valore uguale al proprio

- `compareTo()`: restituisce 0 se sono uguali, < 0 se il

proprio valore è $<$ di quello in ingresso, > 0 altrimenti

6.2 Autoboxing e auto-unboxing

L'autoboxing converte automaticamente un tipo primitivo al suo tipo wrapper associato.

```
Integer k = 3;
Integer[] array = {5,3,7,8}; // autoboxing
int j = k; // k.intValue();
int n = array[j] // equivalente a array[j].intValue();
```

L'auto-unboxing converte automaticamente da un tipo wrapper all'equivalente tipo primitivo.

6.2.1 Nelle stringhe

```
String s = "Mondo"; // String s = new String("Mondo")
String s = "ciao".toUpperCase(); // new String("ciao").toUpperCase();
```

6.3 Costo oggetto memoria

Costa minimo 8 byte.